



Parallel ant colony optimization for vehicle routing with parcel lockers

Phu An Chau^{a,b}, Loan T.T. Nguyen^{a,b}^{*}, Witold Pedrycz^{c,d}, Bay Vo^e

^a School of Computer Science and Engineering, International University, Ho Chi Minh City, Viet Nam

^b Vietnam National University, Ho Chi Minh City, Viet Nam

^c Department of Electrical and Computer Engineering, University of Alberta, Edmonton AB, Edmonton, T6R 2V4, Alberta, Canada

^d Systems Research Institute, Polish Academy of Sciences, Warsaw, 00-901, Poland

^e Faculty of Information Technology, HUTECH University, Ho Chi Minh City, Viet Nam

ARTICLE INFO

Keywords:

VRP
ACO
Multidimensional pheromone matrix
Parcel locker
Parallel computing
Adaptive mechanism

ABSTRACT

The recent surge in Vietnam's e-commerce has significantly strained urban infrastructure, leading to increased last-mile delivery times, higher transportation costs, and diminished customer satisfaction. In response, many countries are adopting sustainable solutions like parcel lockers, which facilitate contactless deliveries. However, integrating parcel lockers introduces a new variant to the classic routing problem: the Vehicle Routing Problem with Parcel Lockers (VRPPL), which demands effective management of multiple delivery options. To address this, this research proposes a novel 3D Enhanced Parallel Ant Colony Optimization (3D-PACO) model. Our core contribution is the introduction of a pioneering multidimensional pheromone matrix to extend the traditional Ant Colony Optimization (ACO) for the VRPPL's multiple delivery options. Further enhancing this, we propose an adaptive mechanism for dynamic ant allocation per thread for economical resource utilization. Finally, we adopt a master-slave parallel schema to the VRPPL enabling deployment of a large total number of ants. The experimental results on various VRPPL benchmark datasets indicate statistically significant improvements in solution quality with tremendous savings in computational runtime of 46.4% to 83.5% across different dataset sizes, compared to the original work.

1. Introduction

To mitigate the urban infrastructure strain, parcel lockers have been considered as a sustainable green solution, and have been adopted in many countries, including China, England, Poland, etc. [1]. However, the integration of parcel lockers, or pickup and drop-off (PUDO) points in general, brings new challenges to the current routing problem. Unlike traditional vehicle routing problems (VRP) in which each customer is associated with a unique delivery point, there can be multiple alternative locations for a customer's request. Furthermore, these delivery options might be preoccupied or capacity-limited. Such challenges demand the routing algorithm not only be able to find near-optimal permutations of customers to be served but also consider the route options.

Ant Colony Optimization (ACO), proposed by Dorigo [2], is a well-established method to solve VRPs, inspired by the foraging behavior of ants. In the ACO system, pheromone trails serve as indirect, distributed numeric communication channels between ants. Our research identifies a gap in the literature where the multidimensional pheromone matrix, which can directly model multiple delivery options for each customer,

remains largely unexplored, therefore, explaining the sparsity of research implementing ACO to solve VRPPL or VRPDO. Furthermore, since adding multiple dimensions increases the search space, sequential ACO models could suffer from expensive computation and memory issues, especially for large-scale instances.

Parallel computing involves distributing computational tasks across multiple processors to reduce execution time, leveraging architectures like multicore CPUs, GPUs, and distributed systems such as Apache Hadoop and Spark [3]. The integration of parallel computing with metaheuristics aims to enhance efficiency, scalability, and solution quality for large-scale problems, addressing limitations of sequential implementations. The inherent population-based nature of ACO makes it inherently suitable for parallel processing [4]. The algorithm operates with multiple sub-colonies of artificial ants, each of which independently explores the solution space. This independence allows for the distribution of the tasks associated with each sub-colony, such as solution construction and evaluation, across multiple processing elements, enabling concurrent execution.

* Corresponding author at: School of Computer Science and Engineering, International University, Ho Chi Minh City, Viet Nam.

E-mail addresses: chuanphu@gmail.com (P.A. Chau), ntloan@hcmiu.edu.vn (L.T.T. Nguyen), wpedrycz@ualberta.ca (W. Pedrycz), vd.bay@hutech.edu.vn (B. Vo).

This research offers three key contributions. First, we propose a novel 3D pheromone matrix that significantly extends the ACO model's applicability to VRPs featuring multiple delivery options, explicitly incorporating customer service preferences as constraints. This ACO model is further enhanced with a local search mechanism to effectively escape local optima. Second, we implement a master-slave parallel architecture designed to efficiently deploy a large population of ants. An elitist update strategy not only reduces pheromone synchronization runtime at the master process but also enhances exploitation, leading to faster convergence and improved solution quality for large-scale problems. Finally, to overcome critical scaling and hyperparameter tuning challenges, we introduce an adaptive mechanism for dynamically adjusting the number of ants per thread. This significantly enhances the model's robustness and ensures economical computational resource utilization.

The article is further organized as follows. Section 2 reviews the problem variants, different approaches to VRPs, ACO extensions, and parallel schema for ACO. Section 3 focuses on presenting the 3D-pheromone matrix and the proposed algorithm. In Section 4, we perform analysis on the model's robustness, scalability, and performance comparison to the published results of Yu et al. [5]. Lastly, Section 5 summarizes the article, states the limitations of the proposed method, and suggests future works.

2. Literature review

2.1. Different variants of VRP

Capacitated VRP (CVRP) is one of the most extensively studied variants. CVRP requires that each customer can only be serviced by one vehicle, and the total demand of the customers assigned to a vehicle should not exceed its capacity. Additionally, there can be constraints related to the number of used vehicles or the maximum traveling distance of each one. Vehicle Routing with Time Windows (VRPTW) introduces additional complexity by assigning a specific time window during which each customer must be serviced. While early arrivals are allowed, they incur waiting times, and late arrivals are penalized. If considered as soft constraints, traveling time along with distance are both targets to be optimized in multi-objective problems [6].

Recent studies have expanded the classic VRP by incorporating multiple delivery options, such as home and locker delivery, and by addressing new challenges posed by parcel lockers. For instance, Tilk et al. [7] introduced the Vehicle Routing Problem with Delivery Options (VRPDO), where customers can choose between home delivery or locker delivery. Grabenschweiger et al. [8] proposed the Vehicle Routing Problem with Heterogeneous Locker Boxes (VRPHLB), which models the heterogeneous sizes and capacities of the lockers. The problem becomes intricate as it requires efficient routing and parcel-packing strategies to ensure that parcels are stored within locker constraints. Despite reflecting real-world logistics with locker heterogeneity and parcel sizing, the research's model lacks preference flexibility.

The Vehicle Routing Problem with Parcel Lockers (VRPPL), introduced by Yu et al. [5], further extends these models by introducing three types of customers: those requesting home delivery (type-1), those requesting locker delivery (type-2), and those who are flexible and can accept either option (type-3). The VRPPL model incorporates time windows and vehicle capacities, while minimizing the total travel distance. The authors extended Solomon's VRPTW instances to create new benchmark datasets for the VRPPL.

The crowdsourcing problem of [9] exhibits similarity with VRPPL due to the incorporation of alternative delivery options and customer types. The authors address these delivery options through an Allocation-Routing Algorithm (ARA). Specifically, type-2 and type-3 customers are assigned to their closest alternative delivery points using K-means. Type-1 customers are handled by a two-level allocation algorithm. Their results demonstrate that total distribution costs can be reduced by recommending customers utilize these alternative delivery points.

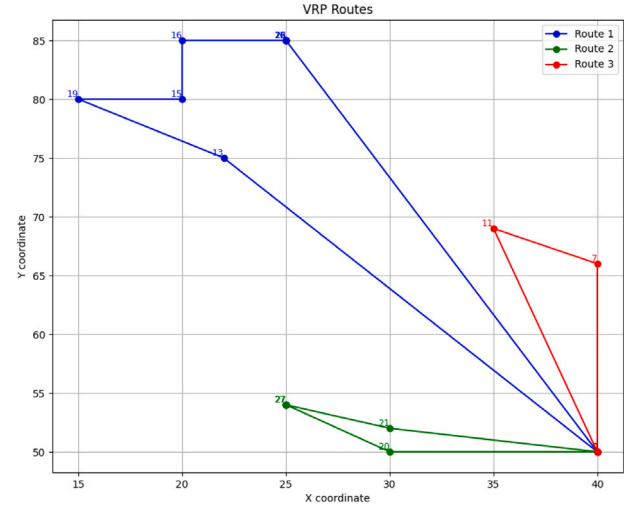


Fig. 1. An example of a VRPPL solution. Nodes 26 and 27 are lockers, and the remaining nodes are customer homes.

2.2. Common approaches to VRPs

Several techniques have been developed to tackle various routing problems and are commonly categorized as exact methods, heuristics, and metaheuristics. Early approaches to the VRP relied on exact optimization techniques such as Mixed-Integer Programming formulations solved via column generation [10] or branch-and-cut algorithms [11]. Despite guaranteeing optimality, exact methods become computationally expensive as problem size grows. Heuristic methods, like the Savings Algorithm [12] and Nearest Neighbor [13], deliver good-quality solutions quickly but are highly dependent on the problem structure and prone to being trapped in local optima.

Meta-heuristic approaches provide a powerful framework for overcoming the limitations of traditional optimization methods for the VRP, guiding the design of effective search strategies that can escape local optima. As classified by Stamadianos et al. [14], these methods fall into two primary categories: Population Search and Local Search. Population Search methods maintain and iteratively update a pool of candidate solutions; this class includes evolutionary algorithms (e.g., Genetic Algorithms) and swarm-based algorithms such as Particle Swarm Optimization (PSO) [15], Artificial Bee Colony (ABC) [16], Cuckoo Search [17] and ACO. Local Search methods, including 2-opt and Tabu Search, intensify the search around a single solution using neighborhood moves.

Despite numerous advancements in meta-heuristic and machine learning, the VRPPL remains a challenging problem due to its complex characteristics. A key bottleneck is the lack of efficient mapping mechanisms to translate customer permutations into delivery options, which is crucial for improving solution quality. Yu et al. [5] introduces probabilistic assignment based on parameter p , and nearest neighbor heuristic insertion for the initial solution. While the advantage of this method relies on its simplicity and model-agnostic nature, it suffers from a significant limitation: the mapping mechanism is not adaptive. This static nature requires extensive tuning for p and can lead to suboptimal solutions. Moreover, the stochastic nature of the method can result in significant variation between runs. Nguyen et al. [9] propose a two-level allocation algorithm, which employs the assignment-first, routing-second mechanism. However, similar to the approach by Yu et al. [5], this two-level mechanism also struggles to address the intricate relationship effectively between delivery options and the corresponding optimal routes (see Fig. 2).

To tackle these challenges, this research presented a multidimensional pheromone matrix as a model-specific mapping mechanism for

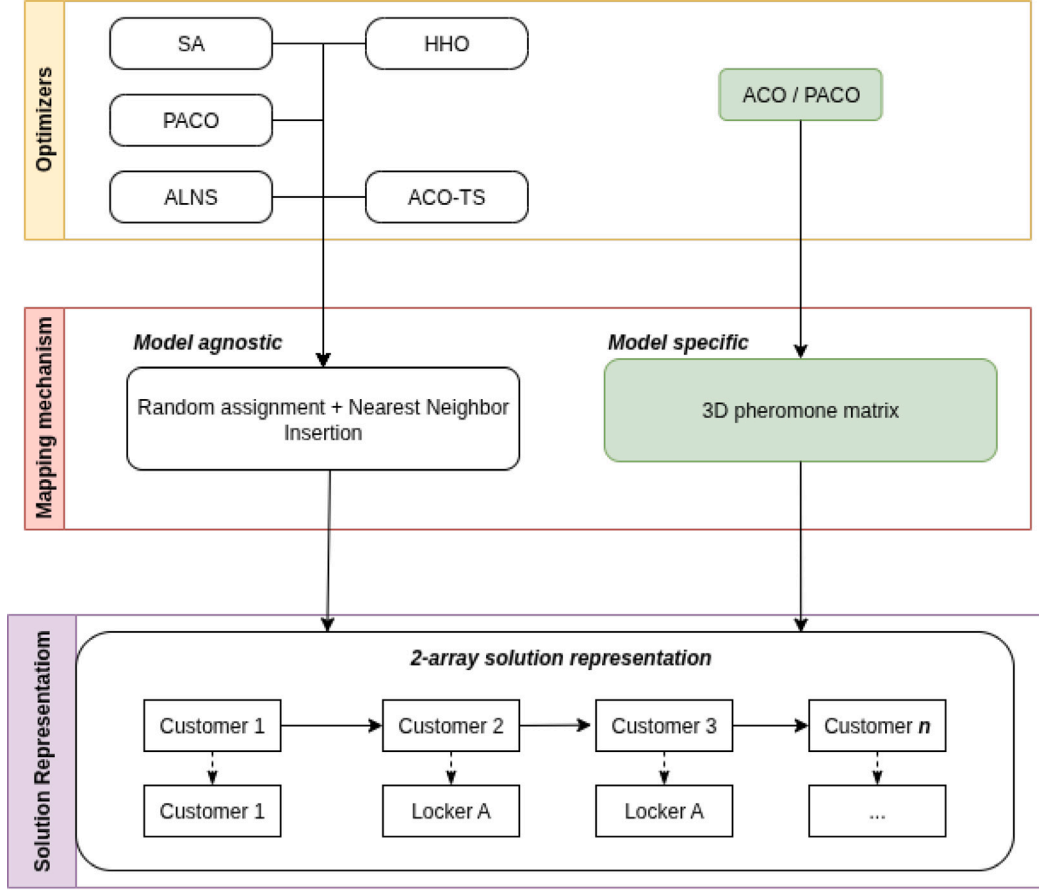


Fig. 2. Meta-heuristic VRPPL approaches: From optimizer to solution representation.

ACO, or any constructive graph-based metaheuristics that can optimize the solution quality by effectively capturing the relationships between customer permutations and their preferred delivery options.

2.3. ACO and its extensions

Based on indirect pheromone trails, the artificial ants in ACO probabilistically construct solutions, and the pheromone trails are subsequently updated through both evaporation and deposition, guided by the optimality of the generated solutions. In ACO, a colony of artificial ants constructs solutions to a combinatorial problem by traversing a graph $G = (N, E)$. Each ant incrementally builds a path by moving from node i to node j with the probability shown in below:

$$P_{i \rightarrow j}^{(k)} = \begin{cases} \frac{[\tau_{ij}]^\alpha [\eta_{ij}]^\beta}{\sum_{u \in \mathcal{N}_i^{(k)}} [\tau_{iu}]^\alpha [\eta_{iu}]^\beta} & \text{if } j \in \mathcal{N}_i^{(k)}, \\ 0 & \text{otherwise,} \end{cases} \quad (1)$$

where:

- τ_{ij} is the pheromone trail intensity on edge (i, j) ,
- $\eta = \frac{1}{d_{ij}}$ is the heuristic desirability (e.g., inverse distance),
- $\alpha, \beta > 0$ control the relative influence of pheromone vs. heuristic,
- $\mathcal{N}_i^{(k)}$ is the feasible neighborhood for ant k at node i .

After all ants complete their tours, pheromone trails are updated via

$$\tau_{ij} \leftarrow (1 - \rho) \tau_{ij} + \sum_{k=1}^m \Delta \tau_{ij}^{(k)}, \quad (2)$$

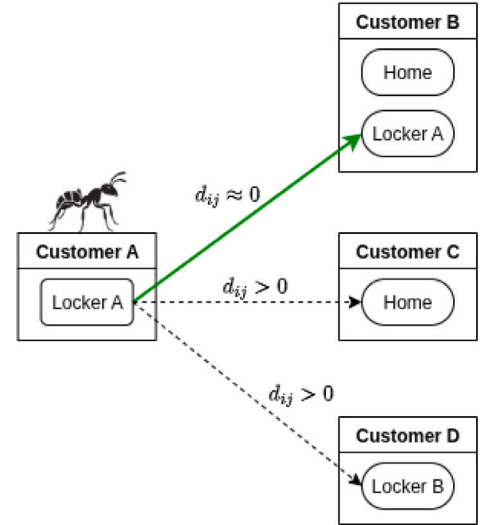


Fig. 3. Demonstration of ACO decision-making process.

$$\Delta \tau_{ij}^{(k)} = \begin{cases} \frac{Q}{L^{(k)}} & \text{if ant } k \text{ used } (i, j), \\ 0 & \text{otherwise,} \end{cases} \quad (3)$$

where $\rho \in (0, 1]$ is the evaporation rate, Q is a pheromone deposit constant, and $L^{(k)}$ is the tour length of the ant k . Eqs. (1) and (2) form the backbone of ACO positive feedback and distributed search mechanisms.

A key advantage of ACO for the VRPPL, when compared to other swarm-based metaheuristics, is its constructive nature and explicit incorporation of heuristic desirability. In this context, desirability is defined as the inverse of the distance between delivery locations. This feature allows ACO to effectively consolidate customers served by the same locker, even during the initial search stages, without heavy reliance on learned pheromone trails. As demonstrated in Fig. 3, since the distance between consecutive demands delivered to the same locker is approximately zero (a small value is added to the denominator to prevent division by zero), the desirability value η_{ij} becomes exceptionally high. This inherent mechanism efficiently guides ants to create compact, consolidated routes from the outset.

An extensive body of research [18] has explored numerous extensions and variants of ACO for addressing both classical routing problems and their increasingly complex modern extensions. The first ACO algorithm is Ant System (AS) [2], followed by Ant Colony System [19] which introduces the indirect pheromone communication. Despite initial encouragement, it was not proven to be competitive against other state-of-the-art methods at that time. Elitist strategy or Elitist Ant System (EAS), mentioned in the same paper, extends AS by only allowing the best solutions to update pheromone trails; such solutions can be in within the current iteration or global-best. $MAX-MIN$ Ant System ($MMAS$) can be considered as a further improvement of EAS, introducing a limitation interval for pheromone value $[\tau_{min}, \tau_{max}] \forall \tau_{ij}$ to balance between exploration and exploitation.

Recent advancements in ACO frequently involve hybridization with other metaheuristics and the incorporation of adaptive strategies. For instance, Ren et al. [20] proposed a hybrid approach where ACO constructs initial solutions, which are then enhanced with adaptive tabu operators, and global pheromone is updated using Simulated Annealing (SA) as a replacement for elitist ants. Similarly, Wan et al. [21] leverage SA to refine solutions generated by elite ACO ants, aiming to escape local optima. Stodola et al. [22] integrate an adaptive pheromone evaporation rate, dynamically adjusted based on information entropy, to maintain population diversity. Similarly, Xue [23] employs an Adaptive ACO (ADACO) for the Multi-Depot Vehicle Routing Problem with Time Windows (MDVRPTW), where the evaporation rate dynamically decreases with iteration count. Finally, Zhou et al. [24] model the Traveling Salesman Problem (TSP) as a Reinforcement Learning (RL) framework and implement Stochastic Gradient Descent (SGD) as an adaptive learning strategy for ACO.

2.4. Parallel computing in ACO

Parallel Ant Colony Optimization (PACO) has proven effective for solving large-scale combinatorial optimization problems such as the VRP. Pedemonte et al. [25] provided an extensive taxonomy of parallel ACO implementations, detailing diverse schemas such as master-slave (coarse-grain, medium-grain, and fine-grain), multi-colony, cellular, parallel independent runs, and hybrid models, each characterized by distinct communication and interaction mechanisms. ACO can be parallelized across various architectures, including multicore-SIMD CPUs [26], cloud computing environments [27], and GPUs [28,29].

One of the sub-categories of master-slave schema is coarse-grain, which assigns complete solution construction and evaluation tasks to individual slave processes, which periodically communicate their results back to a master process that maintains global pheromone information [30]. Its straightforward implementation has shown robust scalability and reduced computational overhead, making it particularly suitable for large-scale VRP instances. Furthermore, shared memory [31] suggests effectiveness in managing global resources between master and slaves, not only reduces communication overhead but also the computational memory required.

Table 1

Notation for parameters and decision variables.

Parameters	
N	Set of all nodes, including depot, customers, and parcel lockers
A	Set of undirected edges connecting nodes
N_{HC}	Set of home delivery customers (type 1)
N_{LC}	Set of locker delivery customers (type 2)
N_{HLC}	Set of flexible customers (type 3)
N_{PL}	Set of parcel lockers
N_C	Set of all customers, $N_{HC} \cup N_{LC} \cup N_{HLC}$
N_{CPL}	Set of customers and parcel lockers, $N_C \cup N_{PL}$
d_i	Demand of customer $i \in N_C$
c_{ij}	Travel cost between nodes i and j
t_{ij}	Travel time between nodes i and j
$[a_i, b_i]$	Time window for service at node $i \in N_{PL}$
s_i	Service duration at node $i \in N_{PL}$
ℓ_{ij}	Binary parameter: 1 if the customer i chooses a locker j , 0 otherwise
ξ_j	Capacity of parcel locker $j \in N_{PL}$ (omitted in our model)
K	Set of vehicles
Q	Maximum capacity of each vehicle
M	A sufficiently large constant for time-dependent constraints
Decision variables	
x_{ijk}	1 if the vehicle k travels from node i to node j , 0 otherwise
h_i	1 if the customer i receives home delivery, 0 otherwise
l_i	1 if the customer i receives locker delivery, 0 otherwise
y_{ij}	1 if the customer i is assigned to a locker j , 0 otherwise
μ_{ik}	Time vehicle k starts service at node i
θ_{k0}	Departure time of vehicle k from the depot
v_k	Time vehicle k completes its final delivery
ψ_{jk}	Total demand loads vehicle k delivers to locker j

3. Methodology

3.1. Mathematical model

The Vehicle Routing Problem with Parcel Lockers (VRPPL) is formulated following the original model of Yu et al. [5] without further extensions. As shown in Table 1, in this problem, a homogeneous fleet of vehicles operates from a central depot to serve customer demands either at their homes or at parcel locker stations. The delivery network is represented as an undirected graph $G = (N, A)$, where N is the set of nodes and A is the set of edges between nodes.

Customers are divided into three types:

1. **Home delivery customers** (type 1) who must receive deliveries at home,
2. **Locker delivery customers** (type 2) who must receive deliveries at a parcel locker,
3. **Flexible customers** (type 3) who can receive deliveries either at home or at a parcel locker.

Parcel lockers are subject to time windows for service and predefined service durations. Each vehicle has a maximum load capacity and must start and end its route at the depot. The objective is to minimize the total travel distance while satisfying vehicle capacity and time-window constraints.

In the study of VRPPL and VRP generally, each delivery node can be annotated by i for departure node or j for the destination node. The depot within the delivery graph G is usually assigned the index 0, $N_0 = \{0\}$. For instance, x_{i0k} will be interpreted as the vehicle k returning to the depot after servicing the customer i . While there can be multiple indices for the depots in the field of Multi-Depot VRP (MD-VRP), this study focuses on the single-depot case. As a result, the customer nodes' index will be denoted as $i, j \in \{1, \dots, n_C\}$ with n_C is the number of customers in N_C .

The main objective is to minimize the total travel distance of all vehicles:

$$D = \sum_{k \in K} \sum_{i \in N} \sum_{j \in N} c_{ij} x_{ijk} \quad (4)$$

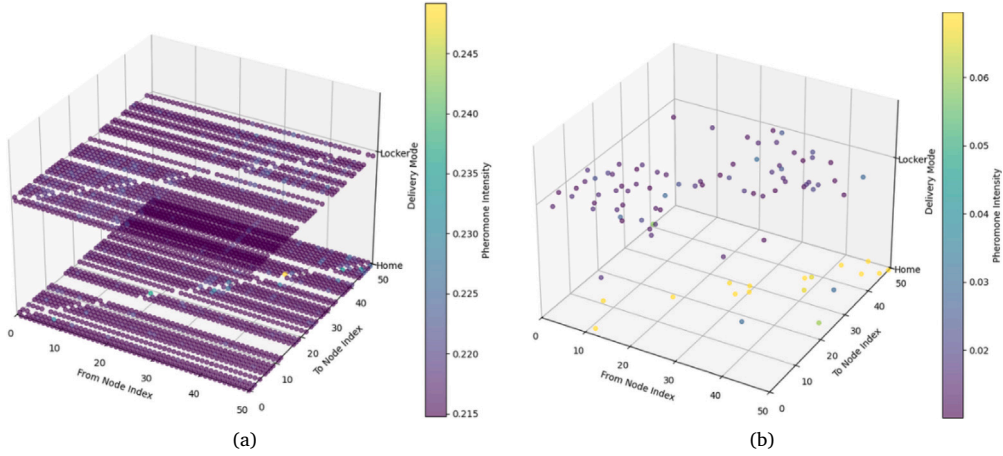


Fig. 4. A visualization of a 3D pheromone matrix: (a) the initial pheromone density and (b) the pheromone density after being converged. The matrix has 3 dimensions: i -axis represents the departure delivery node index ($i \in \{0, \dots, n_C\}$), j -axis represents the destination delivery node index ($j \in \{0, \dots, n_C\}$) and o -axis represents the delivery options at each delivery node ($o \in \{0, 1\}$).

As our study follows the original model [5] without further extensions, we present only the key constraints. Constraints (5)–(10) guarantee that each customer is served exactly once, either at home or at a parcel locker, in accordance with their specified delivery preference:

$$\sum_{k \in K} \sum_{j \in N} x_{ijk} = 1, \quad \forall i \in N_{HC} \quad (5)$$

$$\sum_{k \in K} \sum_{j \in N} x_{ijk} \leq h_i, \quad \forall i \in N_{HLC} \quad (6)$$

$$\sum_{k \in K} \sum_{j \in N} x_{ijk} = 0, \quad \forall i \in N_{LC} \quad (7)$$

$$h_i + l_i = 1, \quad \forall i \in N_C \quad (8)$$

$$h_i = 1, \quad \forall i \in N_{HC} \quad (9)$$

$$l_i = 1, \quad \forall i \in N_{LC} \quad (10)$$

Constraints (11) and (12) ensure that the total load to customer homes and consolidated demands to the lockers (ψ_{jk}) carried by each vehicle k do not exceed its capacity Q :

$$\sum_{k \in K} \psi_{jk} = \sum_{i \in N_{LC} \cup N_{HLC}} y_{ij} \times d_i, \quad \forall j \in N_{PL} \quad (11)$$

$$\sum_{i \in N} \sum_{\substack{j \in N_C \\ i \neq j}} d_j \times x_{ijk} + \sum_{p \in N} \sum_{\substack{m \in N_{PL} \\ p \neq m}} \psi_{mk} x_{pmk} \leq Q, \quad \forall k \in K \quad (12)$$

Constraints (13)–(15) track the arrival time at each visited node, while Constraint (16) ensures that the delivery time windows are respected for home-delivery customers:

$$\partial_{k0} + t_{0i} - \mu_{ik} \leq M(1 - x_{0ik}), \quad \forall k \in K, i \in N_{CPL} \quad (13)$$

$$\mu_{ik} + s_i + t_{ij} - \mu_{jk} \leq M(1 - x_{ijk}), \quad \forall k \in K, i \in N_{CPL}, j \in N_{CPL} \quad (14)$$

$$\mu_{jk} - v_k \leq M(1 - x_{j0k}), \quad \forall j \in N_{CPL}, k \in K \quad (15)$$

$$a_i \times h_i \leq \sum_{k \in K} \mu_{ik} \leq b_i \times h_i, \quad \forall i \in N_{HC}, k \in K \quad (16)$$

Despite the mathematical model of the original paper defines ξ_j as the limited capacity of each locker, the published dataset does not include this information. Therefore, we also omit the locker capacity constraint and primarily focus on operational costs such as vehicle capacity and time windows.

3.2. 3D pheromone matrix

Despite widely applied, the traditional 2D pheromone matrix τ_{ij} cannot distinguish among multiple delivery options for each customer:

it encodes only the desirability of traversing arc (i, j) , not which delivery options (e.g., home, locker type) should be used. [5] addressed this by assigning delivery modes at random, however, this approach has three major drawbacks. Firstly, it requires careful manual tuning of the random-assignment probabilities regarding the problem scale. Secondly, it breaks down when there are more than two delivery options, forcing the model to default to “nearest locker” even if that is suboptimal. Thirdly, it is inherently unstable: the optimal customer permutation may be discovered but still fail to deliver efficiently because the randomly chosen mode may not match the best service option.

Addressing these limitations, this study proposes a 3D pheromone matrix τ_{ijo} , as shown in Fig. 4, that explicitly tracks and learns the efficacy of each delivery option o for the customer j , enabling ants to make jointly optimized routing and mode decisions without manual tuning or restrictive assumptions.

To represent delivery-option effectiveness, the 2D pheromone matrix is extended to a three-dimensional tensor: $\tau_{ijo}, i, j \in N_C, o \in O$. The 3D matrix introduces O as a set of the alternative delivery options (e.g., home delivery, locker 1, locker 2, ...). Each element τ_{ijo} measures the desirability for the vehicle k to travel from the customer i and serve the customer j via option o . Since VRPPL only considers two delivery options (home delivery and nearest locker delivery), we can simplify O to be 0 if home delivery is chosen and 1 if locker delivery is selected. A key advantage of the ACO family in comparison with other swarm-based metaheuristics is its decision interpretability, using the pheromone density, as shown in Fig. 5. The brighter color represents the higher density, thus indicating a higher probability that the ant agent will choose such an arc.

A feasibility mask $M_{ijo} \in \{0, 1\}$ is applied to enforce both home-delivery and locker-delivery constraints. $M_{ij1} = 0, \forall i \in N_C$ If the customer j is type I, and $M_{ij0} = 0, \forall i \in N_C$ if the customer j is type II. Before computing transition probabilities, we zero out all infeasible entries by masking $\tilde{\tau}_{ijo} = \tau_{ijo} \times M_{ijo}$. An advantage of the feasibility mask is that it only needs to be applied once during initialization, since a zeroed-out node will have no probability of being visited.

Both transitional probabilities and the pheromone update function are generalized for a 3D matrix. The generalized transition probability is given by:

$$P_{i \rightarrow j, o}^{(k)} = \begin{cases} \frac{[\tilde{\tau}_{ijo}^{(k)}]^\alpha [\eta_{ijo}]^\beta}{\sum_{u \in \mathcal{N}_i^{(k)}} \sum_{o' \in O} [\tilde{\tau}_{iuo'}^{(k)}]^\alpha [\eta_{iuo'}]^\beta}, & \text{if } j \in \mathcal{N}_i^{(k)} \\ 0, & \text{otherwise} \end{cases} \quad (17)$$

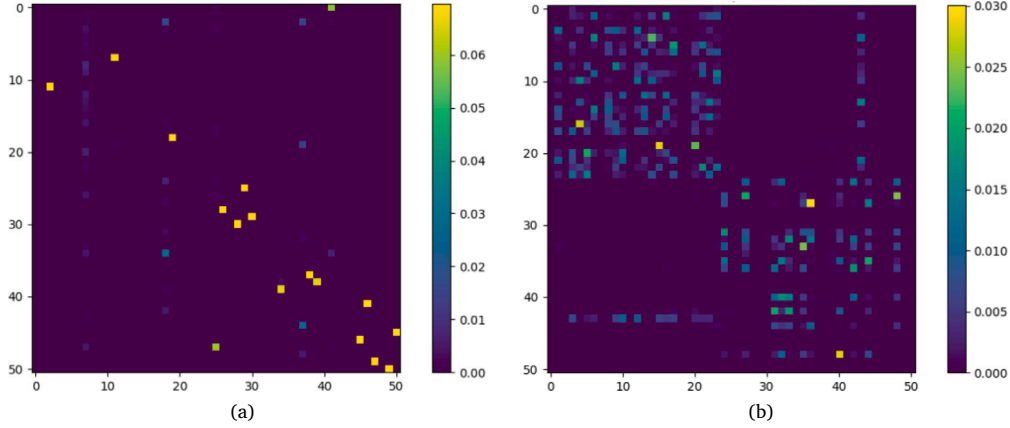


Fig. 5. An illustration of a converged pheromone density by layers: (a) the pheromone density at the home layer $o = 0$ and (b) the converged pheromone density at the locker layer $o = 1$.

Eq. (17) ensures that only feasible arc-option triplets (i, j, o) are considered and that the probabilities across all valid (j, o) sums equal one.

3.3. Solution representation

We denote each demand by the pair (i, o) , where $i \in N_C$ identifies the customer and $o \in O$ specifies the chosen option. Routes are constructed incrementally using a Sequential Insertion Heuristic (SIH). Beginning at the depot ($i = 0$), we select the next demand (j, o) according to the transition probability defined in Eq. (17). Upon inserting each new demand into the route, we update the vehicle's load and the current time. This insertion process continues until adding any remaining demand would violate either the vehicle's capacity or the prescribed time windows. At that point, the route is deemed saturated, and we initiate a new route by resetting both load and time, modeling parallel vehicle deployments.

As illustrated in Fig. 1, any consecutive demands for that locker are aggregated into a single, larger demand. Even though the mathematical model introduced by Yu et al. [5] allows multiple revisits to the same locker by a vehicle, such an action is unlikely to discover more optimal routes. Based on the assumption of the original work, a vehicle will perform consolidated drop-off when visiting a locker. Consequently, upon revisiting without returning to the depot, the vehicle will incur extra mileage for a second trip and consume more of the time budget due to the locker's service time, despite having no packages. As a result, to limit the search space to more promising regions and save the computational runtime, the heuristic strategy of this work treats revisits to the same locker after servicing an intervening location as violations, and hence a new vehicle will be deployed. Finally, if unsatisfied demands remain once all vehicles have been deployed, the overall solution is declared infeasible.

3.4. Coarse-grain parallel schema

Under this 3D representation, the memory complexity is $O(|N|^2 \times |K| \times |O|)$ with $|N| = N_C + N_0$. In practice, since each ant k also carries its tour data, the overall memory footprint grows linearly with the number of ants m , which can become prohibitive for large-scale instances or fine-grained parallelism. To mitigate these challenges, we adopt a coarse-grain master-slave model, as shown in Fig. 6.

In coarse-grain schema, the Master process maintains the global resources, such as the best solution so far, the objective value of such solution, and the shared memory pheromone matrix. Meanwhile, each slave, corresponding to a processing unit, independently constructs complete solutions with objective values, using an enhanced ACO

model. The detailed implementation of the model will be explained in the next section.

Within each iteration, the solutions produced by all worker processes are asynchronously generated and then synchronously aggregated upon their completion. A key mechanism for managing convergence is the stagnation iteration counter (I_{stag}) which is incremented whenever the current iteration fails to discover a new global best solution. Following this aggregation, an elitist strategy is employed: only the top- t ants, identified by their optimal objective values, are permitted to update the global pheromone matrix. By restricting pheromone deposition to this elite subset, our model promotes effective exploitation of the search space while significantly reducing the computational cost associated with pheromone synchronization.

Finally, by utilizing a shared-memory architecture for the global pheromone matrix, the algorithm eliminates the need to distribute copies of the matrix to slave processes. This significantly reduces both memory complexity from $O(m \times |N|^2 \times |O|)$ to $O(|N|^2 \times |O|)$ and the overhead associated with inter-process communication.

3.5. Complete 3D-PACO algorithm for VRPPL

The complete implementation of the proposed 3D-PACO algorithm for the VRPPL is detailed in Algorithm 1. The algorithm begins by initializing the 3D pheromone matrix τ_{ijo} , which encodes the transition desirability from customer i to j under delivery option o . A feasibility mask M_{ijo} is constructed to eliminate invalid transitions. The algorithm maintains a global best solution S^* and its corresponding objective value D^* , initialized to an empty or infeasible solution.

While the stagnation counter does not reach its predefined limit, the algorithm initiates p threads to construct solutions independently. First, the number of ants per thread is adaptively scaled up based on stagnation level, with a maximum increase of 8-fold. This is formalized by $m = \max[m_p * (I_{stag} + 1), 8]$, where m_p represents the initial number of ants per thread. Each ant constructs a complete delivery route using a probabilistic transition rule (Eq. (17)) that combines pheromone values with heuristic desirability. Crucially, infeasible solutions are accepted during this phase to enable a broader exploration of the solution space, recognizing that optimal or near-optimal solutions may emerge from transiently infeasible paths.

The output of ACO is a permutation of served customers and their mapping to delivery choices. However, a high pheromone concentration can lead to ants favoring specific trails, thereby restricting the exploration to particular regions of the solution space and resulting in the generation of repeated or less diverse solutions. To mitigate this limitation, each ACO customer permutation is subsequently explored using local search operations to create more diverse outcomes, with

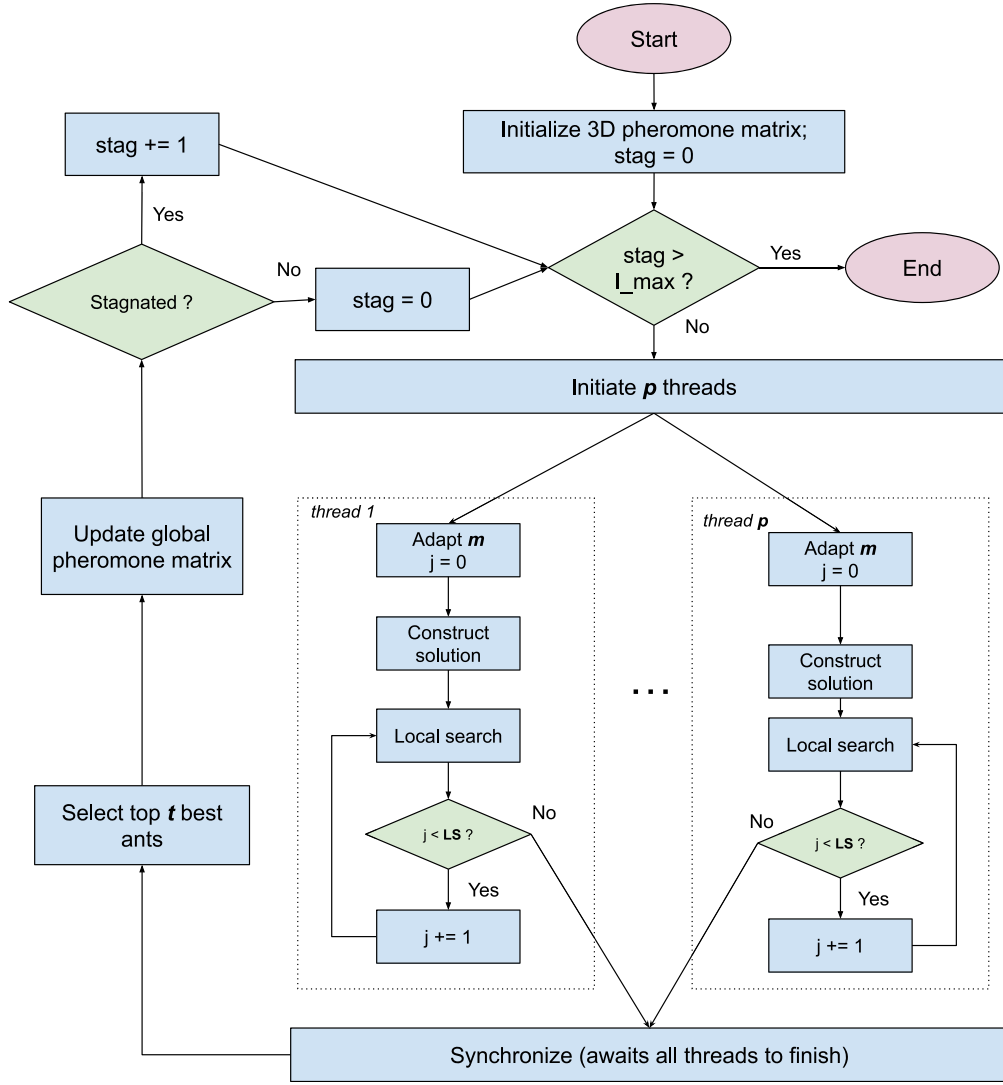


Fig. 6. Coarse-grain parallel model.

the number of neighbors being LS . These local search operations are the same as those employed by Yu et al. [5], comprising insertion, swapping, and inversion.

Once all processors complete their assigned construction tasks, their local solutions are merged into a global solution pool S_{iter} . The master process then selects the top t elite solutions from S_{iter} to perform the global pheromone update using Eq. (19):

$$\Delta\tau_{ijo}^{(k)} = \begin{cases} \frac{Q}{L^{(k)}}, & \text{if } (i, j, o) \in \text{solution path of } k \\ 0, & \text{otherwise} \end{cases} \quad (18)$$

$$\tau_{ijo} \leftarrow (1 - \rho) \cdot \tau_{ijo} + \sum_{k \in t} \Delta\tau_{ijo}^{(k)} \quad (19)$$

If the best solution in the current iteration improves upon D^* , it is saved as the new global best and resets the stagnation counter. Otherwise, it will increase the counter. If terminated, the algorithm ultimately returns the best solution found and its objective value.

4. Experiments and discussion

This section is presented as follows: Section 4.1 presents dataset availability and benchmarks for comparison; Section 4.2 presents computational and the source code repository for reproduction; Section 4.3 explains the hyperparameter tuning method for different optimizers;

Sections 4.4 and 4.5 discuss the robustness and scalability of 3D-PACO, respectively; finally, Section 4.6 compares our performance with the original SA of Yu et al. as well as other methods.

Given that the original work does not benchmark its proposed Simulated Annealing against other meta-heuristic methods, we adapted several others with the same random locker mapping as Yu et al. [5] to ensure consistent comparison. There are other methods, including the Ant Colony Optimization with Tabu Search (ACO-TS) [32], Harris Hawk Optimization [33], Adaptive Large Neighborhood Search (ALNS) [34] and our Parallel Ant Colony Optimization (PACO), but without the multidimensional pheromone matrix. Next, we performed a dedicated parameter tuning for this adapted algorithm using Bayesian Optimization, consistent with the approach used for 3D-PACO.

4.1. Data availability

The datasets used in this research are original benchmark datasets from Yu et al. available at the author's website.¹ These datasets are derived from Solomon CVRP instances, preserving their geographical distribution and customer demands. The author created three sizes: small (25 customers, 2 lockers), medium (50 customers, 3 lockers), and

¹ <https://www.vincentyu.info/data>.

Algorithm 1 3D-PACO Algorithm

Input: Graph $G = (N, E)$
Parameters: $\alpha, \beta, \rho, Q, m_{min}$ (ants per thread), p (processors), t (top elite ants), I_{max} (max stagnation), I_{LS} (local search iterations).
Output: The best solution S^* and the corresponding objective value D^*

Initialize pheromone matrix τ_{ijo}
Construct infeasibility mask M_{ijo}
 $S^* \leftarrow \emptyset, D^* \leftarrow \infty, I_{stag} \leftarrow 0$
while $I_{stag} \leq I_{max}$ **do**
 $S_{iter} \leftarrow \emptyset$
 $m_p \leftarrow m_{min} \cdot \min(I_{stag} + 1, 8)$
 for each processor $j = 1$ **to** p **in parallel do**
 $S_{local}^{(j)} \leftarrow \emptyset$
 for each ant $a = 1$ **to** m_p **do**
 Construct solution S_a using the 3D-ACO transition rule
 for I_{LS} local search iterations **do**
 Apply local search (swap, insert, 2-opt) to S_a
 end for
 Evaluate D_{S_a}
 Add S_a to $S_{local}^{(j)}$
 end for
 Wait for all p processors to finish
 $S_{iter} \leftarrow \bigcup_{j=1}^p S_{local}^{(j)}$
 Sort S_{iter} by D
 Select top t elite solutions from S_{iter}
 Update τ_{ijo} using selected solutions
 Let S_{best} be the best in S_{iter} , D_{best} its cost
 if $D_{best} < D^*$ **then**
 $S^* \leftarrow S_{best}, D^* \leftarrow D_{best}$
 $I_{stag} \leftarrow 0$
 else
 $I_{stag} \leftarrow I_{stag} + 1$
 end if
end while
return S^*, D^*

large (100 customers, 5 lockers). Each size includes 56 testing instances across three distribution types: clustered (C), random (R), and random-clustered (RC). The locker time window is identical to the depot's, and the service time at a locker is half of a customer's. A key assumption made in this study is that lockers have infinite capacity, as the provided datasets do not specify locker constraints.

The detailed computational benchmark results of Section 4.6 are published on Mendeley Data [35]. The dataset includes the best-known solutions, average distances, and runtimes for each method across all instances. This benchmark serves as a reference for future research in VRPPL and related optimization problems.

4.2. Empirical configuration

All algorithms and models were implemented in C++ and are available in our GitHub repository². The model is parallelized with OpenMP to support multithreaded execution.

Experiments were conducted on a computer with an Intel(R) Xeon(R) CPU E5-2696 v3 @ 2.30 GHz, 62 GB of RAM, and 36 threads, achieving a performance of 196.17 GFLOPS. No GPU acceleration was used. The operating system was Ubuntu 24.04.

Table 2
Tuned parameters of 3D-PACO.

	m	I_{max}	LS	p	α	β	ρ	Q	t
Small	10	10	20	64	1.25	1.0	0.5	2.0	40
Medium	30	10	50	64	1.25	1.0	0.5	2.0	40
Large	50	10	100	64	1.25	1.0	0.5	2.0	40

4.3. Hyperparameter tuning

To ensure fair and optimized performance comparisons, the hyperparameters of ACO-TS, ALNS, HHO, and our 3D-PACO model were systematically tuned. Bayesian Optimization (BO) [36] was chosen for this process, as it offers superior computational efficiency compared to exhaustive methods like Grid Search. PACO (without the structured 3D matrix) utilized the same parameters as 3D-PACO.

Given that native C++ implementations for BO are not readily available, we leveraged the scikit-optimize library within a Python environment to manage the optimization routines. Recognizing the substantial computational expense of independently optimizing parameters for all 56 instances within each dataset, we adopted a pragmatic approach: five instances were randomly sampled from each dataset. For every sampled instance, the average objective distance over five independent runs served as the fitness value for the BO algorithm. The number of threads used will always be set to the available maximum ($p = 64$). The tuned set of parameters is shown in Table 2.

4.4. Sensitivity analysis

Next, we employed a One-Factor-At-a-Time (OFAT) approach to analyze the impact of parameter variance across different datasets. Using the BO-tuned parameter set as our baseline, we systematically varied each parameter within a predefined range while keeping all other parameters fixed. The percentage deviation PD (%) was recorded for each variation, reflecting the parameter's influence on the model's performance. For computational efficiency, 5 instances were randomly selected from each dataset, and the experiment was conducted five times on each instance. The performances were aggregated by averaging the results across these runs.

We began with the two parameters that most directly control the search budget and sampling effort: the number of ants per thread (m) and the maximum allowed stagnation iterations (I). Across all instance sizes, these parameters show the clearest trade-off between runtime and solution quality: increasing either m or I raises runtime in an approximately linear manner while providing diminishing returns in solution cost beyond a moderate threshold (see Fig. 7). In practice, this means that modest populations and iteration budgets are sufficient to capture most of the obtainable improvement, whereas overly small values can be harmful. On our medium (see Fig. 8) and large tests (see Fig. 9), we observed dramatic quality degradation when I was set too low (costs doubled when $I < 5$ on medium and when $I < 10$ on large instances), and some cost increases occurred when m dropped below about 30 on large instances. Thus, when tuning for larger problems, it is safer to increase m and I moderately rather than to scale them down to reduce runtime; the observed thresholds are useful starting points for a problem-dependent calibration.

The pheromone-heuristic balance, encapsulated by α (pheromone importance) and β (heuristic importance), exhibits markedly different behaviors: α has strong, dataset-dependent effects, while β is comparatively inert within the ranges we tested. Specifically, very low pheromone influence ($\alpha \leq 0.75$) caused severe runtime penalties and poorer quality on larger instances. Furthermore, runtime increases of roughly 50% to 150% and cost rises of the order of 50% were observed on medium/large sets, and on the large set the runtime curve is non-monotonic with a local minimum near $\alpha = 1.25$. This indicates that pheromone strength interacts with problem scale and search stability:

² <https://github.com/chaunaphu/routing-PL>.

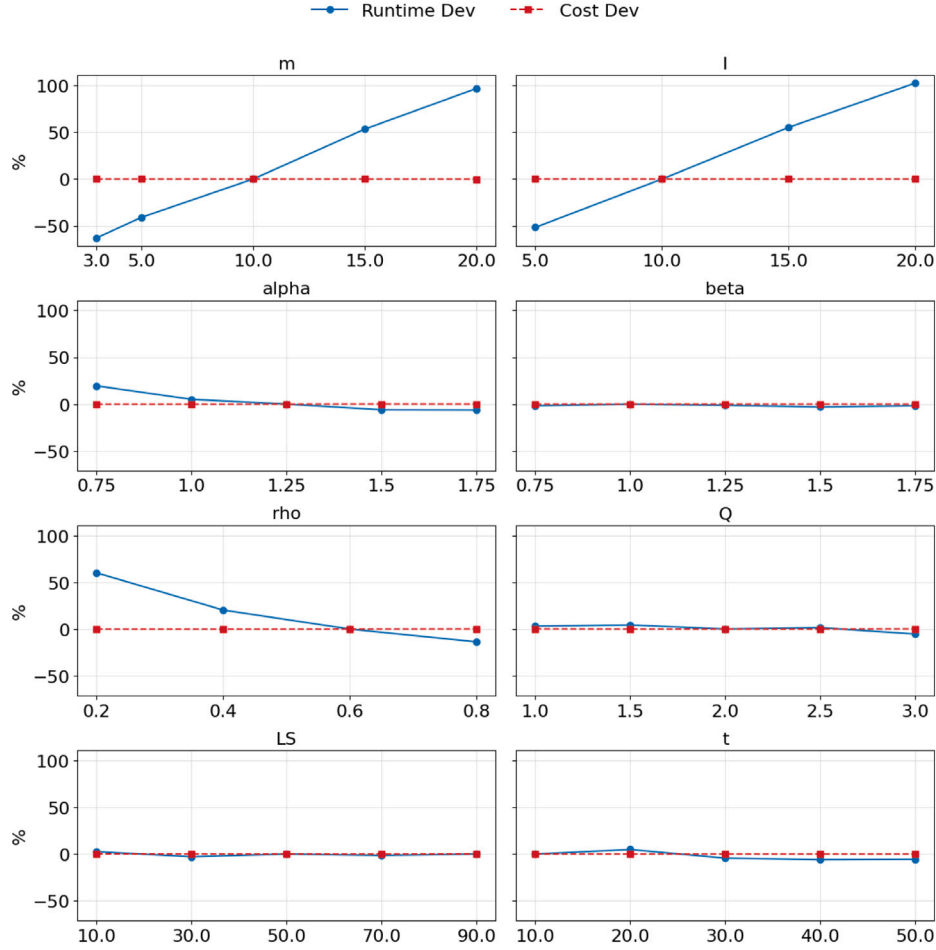


Fig. 7. Sensitivity on the small dataset.

too little pheromone leaves the search overly heuristic-driven and unstable (longer, oscillatory searches), while a moderate pheromone influence helps stabilize convergence. By contrast, β produced only negligible fluctuations in both runtime and cost (typically under 3%), implying that the heuristic used is already informative and that modest changes to its weight do not qualitatively alter behavior. The practical implication is to prioritize careful tuning of α (especially for large problems), using a narrow grid around 1.0–1.5, while keeping β at a reasonable default (e.g., 1–2) and revisiting it only if the heuristic definition changes.

Increasing ρ (faster evaporation) tends to shorten runtime and reintroduce exploration by preventing long-lived reinforcement of suboptimal trails, while very low ρ values permit pheromones to accumulate on poor solutions and can cause premature convergence. Our results show a steady reduction in runtime with higher ρ values, and we observed premature fixation at low values, ρ particularly on the large dataset. Practically, this recommends a moderate-to-high evaporation setting for complex instances (so that the algorithm can escape plateaus), while too-aggressive evaporation should be avoided because it can wash out valuable learned information too quickly.

Two parameters that had comparatively small effects across the tested ranges are the pheromone deposit constant Q and the local-search (LS) frequency. Variations in Q produced only small runtime and cost fluctuations, suggesting that the absolute scale of pheromone updates matters less than the relative distribution of pheromone across edges; therefore, Q may be set proportional to the objective magnitude and need not be exhaustively tuned. Local search showed only small sensitivity as well, but this hides an important conditional observation: the cost of invoking LS matters. When LS is cheap relative to

the iteration cost, frequent use yields net gains; when expensive, it increases runtime with modest benefit. Thus, the sensible rule is to use inexpensive or lightweight LS variants more aggressively and reserve heavy LS for occasional refinement of elite solutions.

4.5. Scalability analysis

Model scalability will be assessed based on speedup (relative to its sequential version ($p = 1$)), and resource utilization efficiency across varying thread counts (8, 16, 32 and 64). The number of ants per thread m is adjusted proportionally ($m = \text{total ants}/p$) to maintain a consistent global ant population. For instance, with a large-scale dataset and 32 threads, $m = 3200/32 = 100$.

$$S(p) = T(1)/T(p) \quad (20)$$

$$E(p) = S(p)/p = T(1)/(p \times T(p)) \quad (21)$$

where $T(p)$ is the runtime (measured in seconds) with p threads.

Fig. 10 offers insights into the scalability of the adaptive parallel ACO model, revealing distinct behaviors based on problem size. Fig. 10(a) demonstrates that for small problem instances, gains quickly saturate, peaking around 7–8 $\times$ speedup with modest thread counts (approx. 16–20). This rapid plateau is attributed to the increasing dominance of parallelization overheads, such as communication and pheromone update, over the limited workload. For medium instances, speedup is more sustained, reaching about 15 $\times$ at 64 threads, indicating better resource utilization. However, the model's true strength in scalability

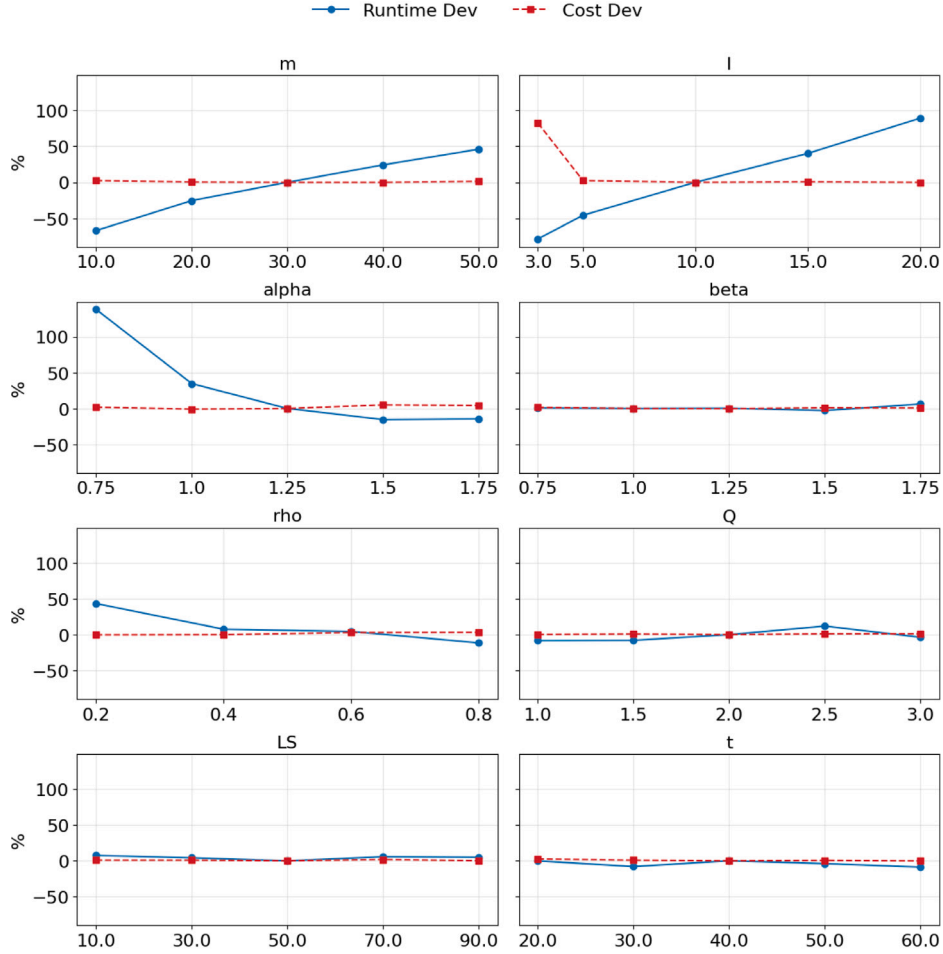


Fig. 8. Sensitivity on medium dataset.

is evident with large instances, demonstrating robust scalability, especially for large datasets, achieving over 20× speedup with 64 threads despite sublinear scaling.

Fig. 10(b) further clarifies these trends. For small problems, efficiency sharply declines, confirming the poor utilization of additional threads. Medium problems show a more gradual efficiency drop. Critically, large problems maintain significantly higher efficiency across the entire range of thread counts, reflecting a much more effective and economical use of parallel computing resources. These findings collectively establish the proposed adaptive parallel ACO as a highly promising solution for large-scale, computationally intensive VRPPL challenges. This indicates that despite the model's speedup is sub-linear, 3D-PACO still displays remarkable runtime improvement.

These findings collectively establish the proposed adaptive parallel ACO as a highly promising solution for large-scale, computationally intensive VRPPL challenges. This indicates that despite the model's speedup being sublinear, 3D-PACO still displays remarkable runtime improvement.

4.6. Computational results

In this experiment, we measure the Wall-Clock Time as the primary standard for the solver's runtimes. Despite CPU Time being the benchmark to compare the algorithmic intensity across different algorithms, we argue that logistics stakeholders require near real-time response from the solver for making immediate actions; thus, providing the CPU Time may cause misleading impressions.

4.6.1. Statistical test against *yu et al.*

Since the distance and runtime may vary significantly within each dataset, the improvement of our 3D-PACO compared to the original SA method will be calculated as percent deviation (PD) of each instance:

$$PD_{(BKS)} (\%) = \frac{D_{3D-PACO}^{best} - D_{SA}^{best}}{D_{SA}^{best}} \times 100\% \quad (22)$$

$$PD_{(D)} (\%) = \frac{D_{3D-PACO}^{avg} - D_{SA}^{avg}}{D_{SA}^{avg}} \times 100\% \quad (23)$$

$$PD_{(T)} (\%) = \frac{T_{3D-PACO} - T_{SA}}{T_{SA}} \times 100\% \quad (24)$$

For the statistical comparison of model performances, we first conduct a Shapiro-Wilk test for normality on the differences between the paired observations to assess their distributional symmetry. Subsequently, a Wilcoxon signed-rank test or a paired t-test will be selected based on the outcome of the normality test, with a significance level of $\alpha = 0.05$. The primary objective is to test whether our proposed 3D-PACO model significantly outperforms SA:

- $H_0 : \hat{P}D_{metric} = 0$
- $H_a : \hat{P}D_{metric} < 0$

where $\hat{P}D_{metric} = \frac{\sum \hat{P}D_{(metric)}}{N} (\%)$ represents the average performance difference between 3D-PACO and SA. A negative value indicates that 3D-PACO achieved a better performance for the given metric.

As shown in Table 3, for small instances, 3D-PACO, while achieving lower average distance and BKS, did not exhibit a statistically significant advantage over SA on these metrics, respectively, by both

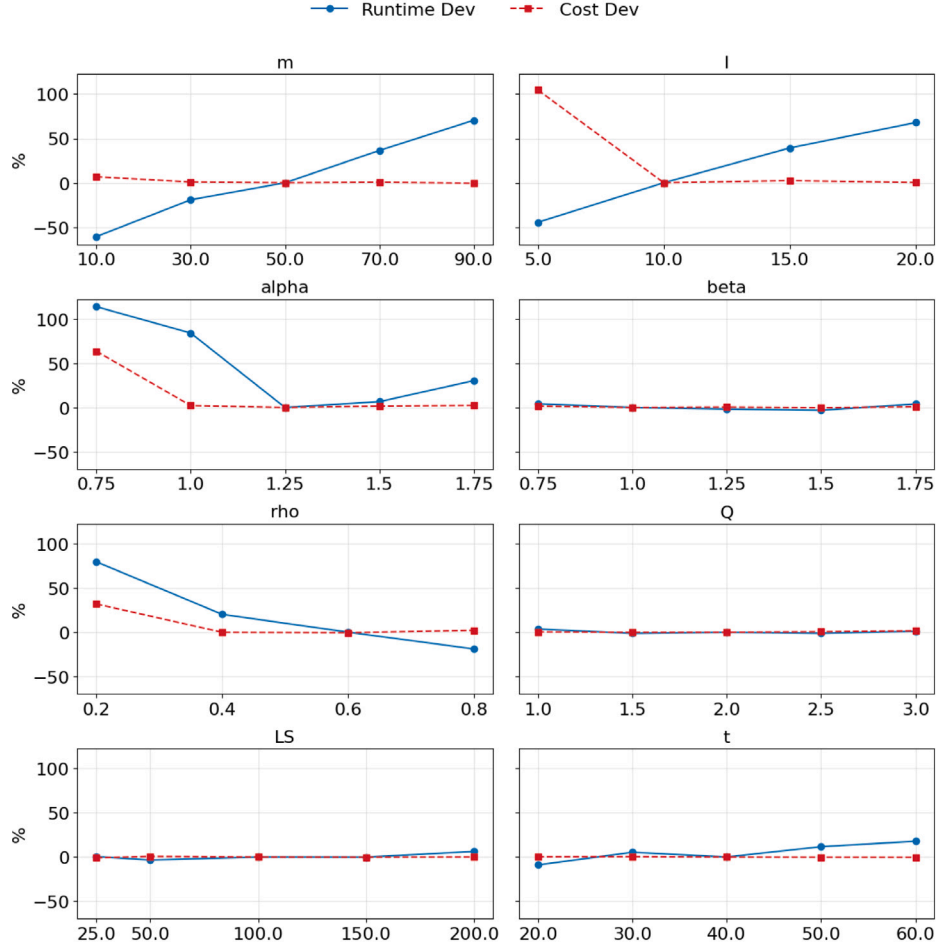


Fig. 9. Sensitivity on large dataset.

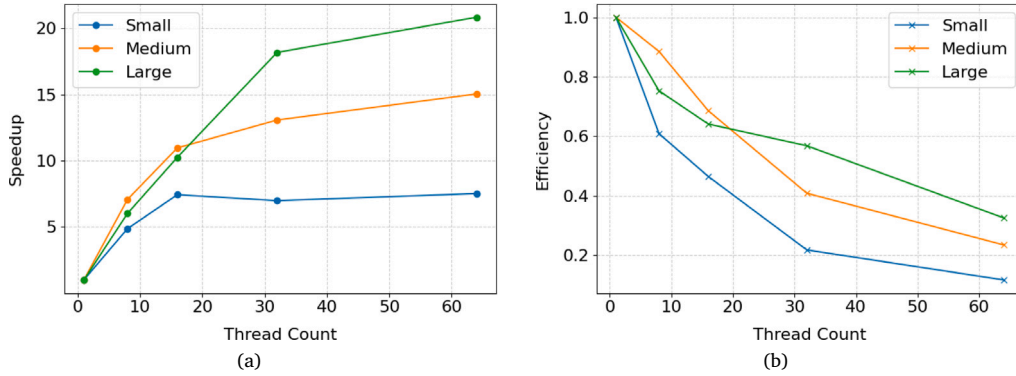


Fig. 10. Scalability Analysis: (a) Speedup and (b) Efficiency across data sizes.

pair t-test and Wilcoxon test. However, PACO achieves statistically significant runtime reduction by 46.4% and $p \approx 0$.

The performance differentials became more pronounced with increasing dataset complexity. On medium datasets, 3D-PACO significantly surpassed SA in terms of finding the best routes (-1.3%), and remarkably reduced wall-clock time by 83.52%. The average distance for the medium dataset remained non-significant.

The most compelling performance advantages for 3D-PACO were observed on large datasets. Here, our model consistently delivered

statistically significant improvements across all evaluated metrics: a lower average distance (-0.94% , $p = 0.049$), superior BKS (-2.65%) with half of the runtime (-49.96%). T-tests were used for BKS and Avg D, while Wilcoxon was used for Avg T on large datasets.

Considering the overall aggregated performance, 3D-PACO maintained its significant advantage in average runtime ($\hat{P}D = -59.96\%$ p -value 0.001). However, the overall best-known solution and average distance differences, while numerically favoring 3D-PACO, did not achieve statistical significance (p -values 0.772 for BKS and 0.077 for

Table 3
Summary of statistical test results.

Dataset	Metric	Avg. PD	Normality test (<i>p</i> -value)	Statistical test	<i>p</i> -value	Conclusion
Small	BKS	-0.68	0.0000	Wilcoxon	0.7721	Not significant
	Avg D	-0.51	0.0000	Wilcoxon	0.8178	Not significant
	Avg T	-46.39	0.3680	T-test	0.0000	Significant
Medium	BKS	-1.30	0.0000	Wilcoxon	0.0005	Significant
	Avg D	-0.53	0.0000	Wilcoxon	0.9870	Not significant
	Avg T	-83.52	0.0150	Wilcoxon	0.0000	Significant
Large	BKS	-2.65	0.5120	T-test	0.0000	Significant
	Avg D	-0.94	0.5080	T-test	0.0490	Significant
	Avg T	-49.96	0.0000	Wilcoxon	0.0000	Significant
Overall	BKS	-1.54	0.0000	Wilcoxon	0.0000	Significant
	Avg D	-0.66	0.0000	Wilcoxon	0.0770	Not significant
	Avg T	-59.96	0.0000	Wilcoxon	0.0000	Significant

Denotes the significant difference exists.

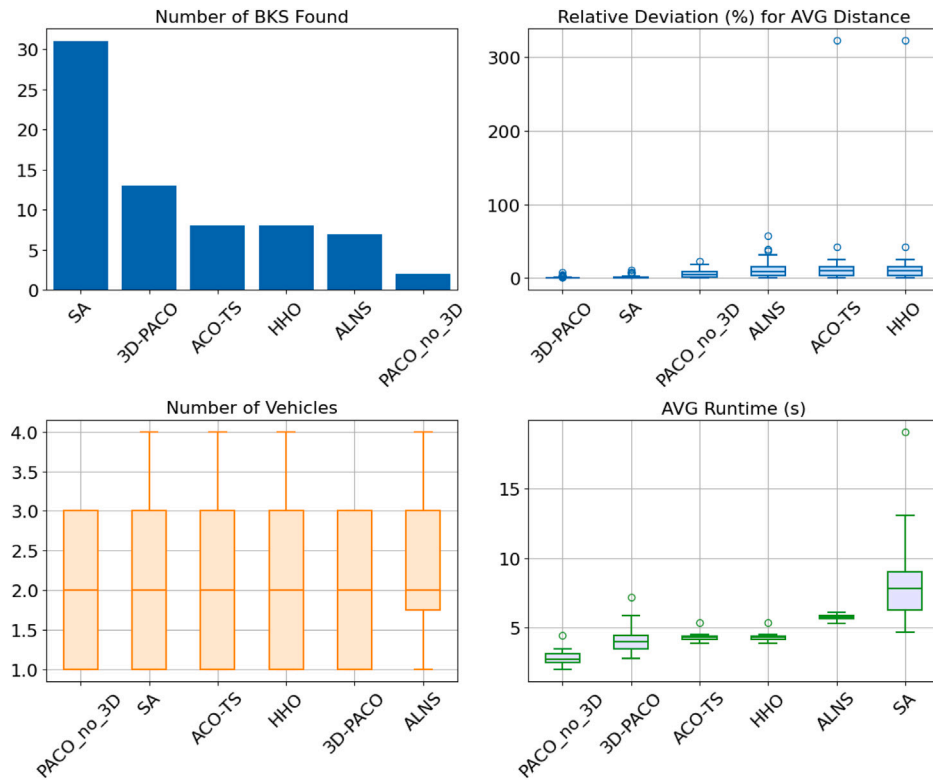


Fig. 11. Benchmark comparison on the small dataset.

average distance, respectively), suggesting that while 3D-PACO generally performs better in quality, this improvement is not consistently significant across all dataset sizes when aggregated.

4.6.2. Benchmark comparison

To evaluate the effectiveness of the proposed 3D-PACO, we compared it against its non-3D counterpart (PACO), as well as four baselines: SA, ACO-TS, ALNS, and HHO. Performance was assessed using three metrics: number of best-known solutions (BKS) recovered, relative deviation from best average distance, and computational efficiency in terms of runtime. Figs. 11–13 illustrate the results across the small, medium, and large benchmark datasets, while the aggregated statistics are summarized in Table 4.

On the small dataset (Fig. 11), SA maintains its dominance in terms of BKS discovery, recovering 31 out of 56 best-known solutions, followed by 3D-PACO with 13. The other metaheuristics trail behind, with ACO-TS, ALNS, and HHO each solving roughly eight instances, and PACO without the 3D pheromone matrix solving only

two. Despite SA's clear strength in finding the best solutions, the PACO family demonstrates remarkable consistency in average performance: the mean distance produced by 3D-PACO is slightly better than SA, and PACO (no 3D) remains competitive against ALNS, ACO-TS, and HHO. Where PACO clearly outperforms is in efficiency: 3D-PACO completes in just over four seconds on average, while PACO without 3D achieves the fastest runtime at 2.8 s. In contrast, SA is significantly slower and less predictable, ranging from 4 to 18 s depending on the instance. Vehicle usage was generally stable across methods, averaging two vehicles, but PACO solutions showed tighter distributions.

When moving to the medium dataset (Fig. 12), 3D-PACO emerges as the clear leader across all metrics. It discovers 32 BKS, the highest among all solvers, and delivers the lowest average distance (382.6). More importantly, 3D-PACO demonstrates strong runtime efficiency, solving instances in an average of 26 s, substantially faster than SA (166 s), HHO (191 s), and ALNS (103 s). PACO without 3D remains competitive in terms of runtime (52 s) but fails to achieve strong solution quality, recovering only three BKS and yielding worse average

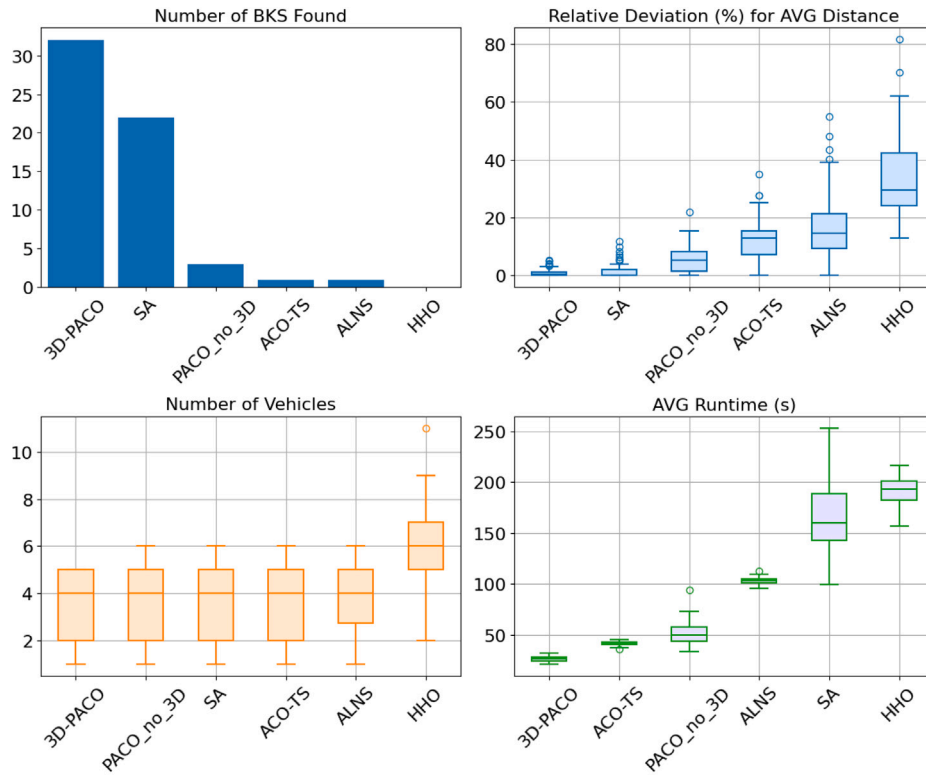


Fig. 12. Benchmark comparison on the medium dataset.

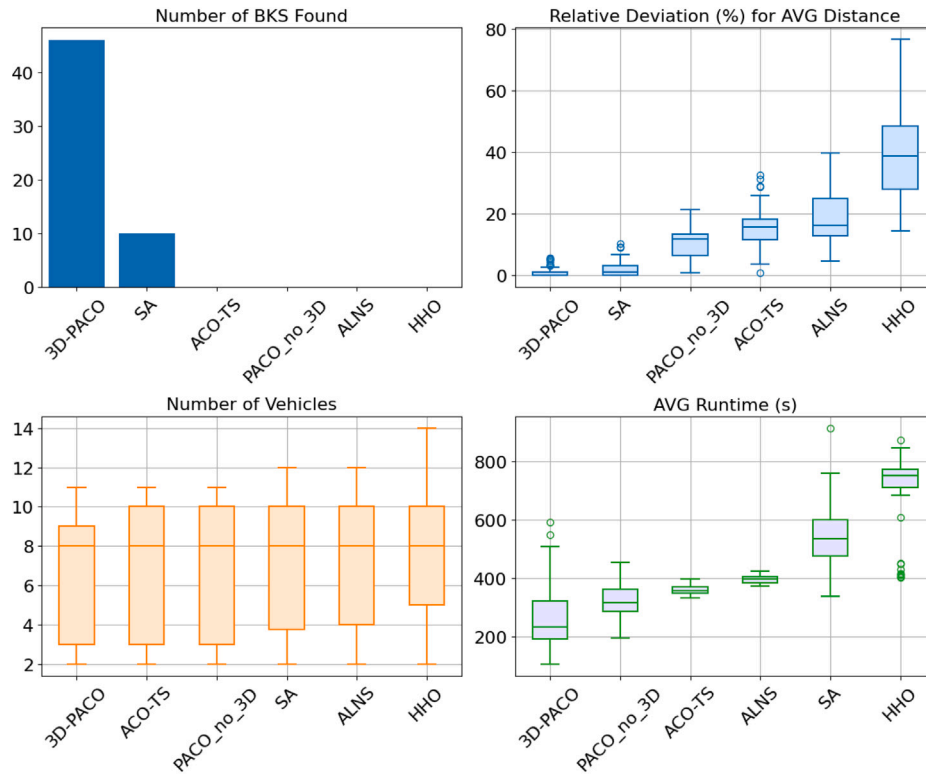


Fig. 13. Benchmark comparison on the large dataset.

distances. The performance gap between PACO-family methods and the rest widens as problem scale increases, highlighting the importance of structured pheromone representation for handling larger neighborhoods efficiently. Vehicle usage results further confirm this advantage:

3D-PACO consistently identifies routes with fewer vehicles compared to ALNS and HHO, demonstrating better packing and assignment quality.

On the large dataset (Fig. 13), 3D-PACO solidifies its superiority, recovering 46 BKS out of 56 and maintaining the best average distance

Table 4
Aggregated performances across datasets.

Methods	Metrics	Large	Medium	Small
3D-PACO	Runtime (s)	266.97	26.08	4.08
	CPU time (s)	17,086.08	1669.12	261.12
	Average distance	693.17	382.55	214.44
	# Best distance found	46	32	13
SA	Runtime (s)	546.40	165.73	8.09
	CPU time (s)	546.40	165.73	8.09
	Average distance	702.80	384.82	215.89
	# Best distance found	10	22	31
PACO (no 3D)	Runtime (s)	322.59	51.82	2.81
	CPU time (s)	20,645.76	3316.48	179.84
	Average distance	756.52	400.40	225.93
	# Best distance found	3	3	2
ACO-TS	Runtime (s)	359.21	41.34	4.31
	CPU time (s)	359.21	41.34	4.31
	Average distance	792.31	427.63	241.19
	# Best distance found	0	1	8
ALNS	Runtime (s)	395.90	103.46	5.75
	CPU time (s)	395.90	103.46	5.75
	Average distance	808.37	440.76	236.52
	# Best distance found	0	1	7
HHO	Runtime (s)	702.69	191.23	4.31
	CPU time (s)	702.69	191.23	4.31
	Average distance	947.69	512.61	241.19
	# Best distance found	0	0	8

Bold indicates the best performance among methods for each metric.

(693.2). SA performs relatively well, solving 10 instances, but suffers from prohibitively high runtimes (over 540 s on average) and much higher deviation from the best distances. Other methods, including ACO-TS, ALNS, and HHO, fail to scale: none discover more than a handful of BKS, and their average distances are substantially worse (all above 790). Runtime comparisons reinforce this trend: 3D-PACO remains efficient at scale, averaging 267 s, which is an order of magnitude faster than SA and significantly better than HHO (703 s). PACO without 3D shows a slightly faster runtime than SA but still lags well behind in solution quality, reinforcing that the 3D matrix structure is not merely an incremental addition but a decisive factor in scaling solution quality and stability.

Taken together as shown in Table 4, the results indicate that 3D-PACO offers a balanced and scalable solution strategy. In small instances, it is slightly behind SA in terms of best-solution discovery but already competitive in average solution quality and markedly superior in runtime. As the problem size grows, 3D-PACO surpasses SA and all other solvers decisively, combining robustness in discovering BKS with consistently lower average distances and highly efficient runtimes. The comparison between 3D-PACO and PACO without the 3D matrix makes clear that the novel multidimensional pheromone representation is responsible for escaping suboptimal solutions while providing timely responses, confirming the design motivation of our proposed method.

However, despite 3D-PACO and PACO outperforming other solvers in terms of wall-clock time, their architecture relies on mobilizing numerous threads ($p = 64$). This leads to the average CPU Time, which is estimated by $AVG(T_{CPU}) \approx AVG(T) \times p$, displaying tremendous computational effort on different datasets. As a result, the benchmark in Table 4 suggests that the implementation of master-slave coarse-grain architecture achieves an advantage in reducing runtime by exchanging for significant costs for algorithmic intensity.

5. Conclusion

This work addressed the Vehicle Routing Problem with Parcel Lockers (VRPPL), a challenging variant of VRP due to multiple delivery modes and operational constraints. We introduced 3D-PACO, a parallel ACO model with a multidimensional pheromone matrix, designed to

optimize routing and locker assignment simultaneously. Experimental results prove that 3D-PACO consistently reduces wall-clock time and indicate improved quality on larger datasets compared to Simulated Annealing [5] and other metaheuristics, while improvements on small datasets are not statistically significant.

Despite these advantages, the multidimensional pheromone matrix remains tied to the ACO framework and incurs additional computational overhead when synchronizing. Furthermore, due to expanding the search space to optimize assignment and routing simultaneously through the 3D pheromone matrix, 3D-PACO requires tremendous computational effort despite parallelism. Future research should explore approaches to optimize the energy efficiency without sacrificing the runtime and solution optimality.

Lastly, a key limitation of VRPPL's dataset is the lack of locker capacity information. Future work should address this limitation to ensure the practical application and investigate its application in multi-objective scenarios [6]. In terms of algorithm improvement, potential directions include exploring asynchronous parallelization (e.g., multi-colony models) to reduce synchronization costs and extending the model-agnostic mapping mechanism to other optimization frameworks.

CRediT authorship contribution statement

Phu An Chau: Writing – original draft, Software. **Loan T.T. Nguyen:** Writing – original draft, Validation, Supervision, Methodology, Conceptualization. **Witold Pedrycz:** Writing – review & editing, Validation. **Bay Vo:** Writing – review & editing, Validation, Methodology.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used Gemini 2.5 Flash in order to assist in improving coherence and grammar accuracy. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Role of the funding source

Please disclose any funding sources and their role, if any, in the writing of the manuscript or the decision to submit it for publication. Examples of involvement include: data collection, analysis, or interpretation; trial design; patient recruitment; or any aspect pertinent to the study. Please also comment whether you have been paid to write this article by a pharmaceutical company or other agency. The information provided here must match the role of the funding source statement in the manuscript. If you are the corresponding author, please state that authors were not precluded from accessing data in the study, and they accept responsibility to submit for publication.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- [1] T.T. Huong, B.N. Thiet, Smart locker-a sustainable urban last-mile delivery solution: Benefits and challenges in implementing in Vietnam, in: 12th NEU-KKU International Conference Socio-Economic and Environmental Issues in Development, 2020.
- [2] M. Dorigo, V. Maniezzo, A. Colnari, Ant system: optimization by a colony of cooperating agents, *IEEE Trans. Syst. Man Cybern. B* 26 (1) (1996) 29–41.
- [3] E. Alhenawi, R.A. Khurma, A.A. Sharieh, O. Al-Adwan, A. Al Shorman, F. Shannaq, Parallel ant colony optimization algorithm for finding the shortest path for mountain climbing, *IEEE Access* 11 (2023) 6185–6196.
- [4] B.A. de Melo Menezes, H. Kuchen, F. Buarque de Lima Neto, Parallelization of swarm intelligence algorithms: literature review, *Int. J. Parallel Program.* 50 (5) (2022) 486–514.
- [5] F.Y. Vincent, H. Susanto, P. Jodiawan, T.-W. Ho, S.-W. Lin, Y.-T. Huang, A simulated annealing algorithm for the vehicle routing problem with parcel lockers, *IEEE Access* 10 (2022) 20764–20782.
- [6] W. Chen, Y. Liu, J. Liu, The multi-objective vehicle routing problems with parcel lockers for simultaneous pick-up and delivery, in: 2024 IEEE Congress on Evolutionary Computation, CEC, IEEE, 2024, pp. 1–8.
- [7] C. Tilk, K. Olkis, S. Irnich, The last-mile vehicle routing problem with delivery options, *OR Spectrum* 43 (4) (2021) 877–904.
- [8] J. Grabenschweiger, K.F. Doerner, R.F. Hartl, M.W. Savelsbergh, The vehicle routing problem with heterogeneous locker boxes, *Cent. Eur. J. Oper. Res.* 29 (2021) 113–142.
- [9] N.N.T. Nguyen, N. Van Hop, Hierarchical allocation-routing heuristic algorithm for crowd-shipping problem with time windows, transshipment nodes, and delivery options, *Expert Syst. Appl.* 268 (2025) 126325.
- [10] T.I. Faiz, C. Vogiatzis, M. Noor-E-Altam, A column generation algorithm for vehicle scheduling and routing problems, *Comput. Ind. Eng.* 130 (2019) 222–236.
- [11] X. Zhang, L. Chen, M. Gendreau, A. Langevin, A branch-and-cut algorithm for the vehicle routing problem with two-dimensional loading constraints, *European J. Oper. Res.* 302 (1) (2022) 259–269.
- [12] G. Clarke, J.W. Wright, Scheduling of vehicles from a central depot to a number of delivery points, *Oper. Res.* 12 (4) (1964) 568–581.
- [13] G. Laporte, S. Ropke, T. Vidal, Chapter 4: Heuristics for the vehicle routing problem, in: *Vehicle Routing: Problems, Methods, and Applications*, Second Edition, SIAM, 2014, pp. 87–116.
- [14] T. Stamadianos, A. Taxisidou, M. Marinaki, Y. Marinakis, Swarm intelligence and nature inspired algorithms for solving vehicle routing problems: a survey, *Oper. Res.* 24 (3) (2024) 47.
- [15] Y. Marinakis, M. Marinaki, A. Migdalas, A multi-adaptive particle swarm optimization for the vehicle routing problem with time windows, *Inform. Sci.* 481 (2019) 311–329, <http://dx.doi.org/10.1016/j.ins.2018.12.086>.
- [16] D. Trachanatzi, M. Rigakis, M. Marinaki, Y. Marinakis, N. Matsatsinis, Distance related: a procedure for applying directly artificial bee colony algorithm in routing problems, *Soft Comput.* 24 (2020) 9071–9089.
- [17] G.-H. Wu, C.-Y. Cheng, P. Pourhejazy, B.-L. Fang, Variable neighborhood-based cuckoo search for production routing with time window and setup times, *Appl. Soft Comput.* 125 (2022) 109191, <http://dx.doi.org/10.1016/j.asoc.2022.109191>.
- [18] M. Dorigo, T. Stützle, Ant colony optimization: overview and recent advances, *Handbook of Metaheuristics*, vol. 272, 2018, pp. 311–351.
- [19] M. Dorigo, L.M. Gambardella, Ant colony system: a cooperative learning approach to the traveling salesman problem, *IEEE Trans. Evol. Comput.* 1 (1) (1997) 53–66.
- [20] T. Ren, T. Luo, B. Jia, B. Yang, L. Wang, L. Xing, Improved ant colony optimization for the vehicle routing problem with split pickup and split delivery, *Swarm Evol. Comput.* 77 (2023) 101228.
- [21] F. Wan, H. Guo, W. Pan, J. Hou, S. Chen, A mathematical method for solving multi-depot vehicle routing problem, *Soft Comput.* 27 (21) (2023) 15699–15717.
- [22] P. Stodola, J. Nohel, Adaptive ant colony optimization with node clustering for the multipoint vehicle routing problem, *IEEE Trans. Evol. Comput.* 27 (6) (2022) 1866–1880.
- [23] S. Xue, An adaptive ant colony algorithm for crowdsourcing multi-depot vehicle routing problem with time windows, *Sustain. Oper. Comput.* 4 (2023) 62–75.
- [24] Y. Zhou, W. Li, X. Wang, Y. Qiu, W. Shen, Adaptive gradient descent enabled ant colony optimization for routing problems, *Swarm Evol. Comput.* 70 (2022) 101046, <http://dx.doi.org/10.1016/j.swevo.2022.101046>.
- [25] M. Pedemonte, S. Nesmachnow, H. Cancela, A survey on parallel ant colony optimization, *Appl. Soft Comput.* 11 (8) (2011) 5181–5197, <http://dx.doi.org/10.1016/j.asoc.2011.05.042>.
- [26] Y. Zhou, F. He, N. Hou, Y. Qiu, Parallel ant colony optimization on multi-core SIMD CPUs, *Future Gener. Comput. Syst.* 79 (2018) 473–487, <http://dx.doi.org/10.1016/j.future.2017.09.073>.
- [27] J. Peake, M. Amos, N. Costen, G. Masala, H. Lloyd, PACO-VMP: Parallel ant colony optimization for virtual machine placement, *Future Gener. Comput. Syst.* 129 (2022) 174–186, <http://dx.doi.org/10.1016/j.future.2021.11.019>.
- [28] J.M. Cecilia, A. Llanes, J.L. Abellán, J. Gómez-Luna, L.-W. Chang, W.-M.W. Hwu, High-throughput ant colony optimization on graphics processing units, *J. Parallel Distrib. Comput.* 113 (2018) 261–274, <http://dx.doi.org/10.1016/j.jpdc.2017.12.002>.
- [29] R. Skinderowicz, Implementing a GPU-based parallel MAX-MIN ant system, *Future Gener. Comput. Syst.* 106 (2020) 277–295, <http://dx.doi.org/10.1016/j.future.2020.01.011>.
- [30] K.F. Doerner, R.F. Hartl, G. Kiechle, M. Lucka, M. Reimann, Parallel ant systems for the capacitated vehicle routing problem, in: *Evolutionary Computation in Combinatorial Optimization: 4th European Conference, EvoCOP 2004, Coimbra, Portugal, April 5-7, 2004. Proceedings 4*, Springer, 2004, pp. 72–83.
- [31] P. Delisle, M. Gravel, M. Krajecki, C. Gagné, W.L. Price, Comparing parallelization of an ACO: message passing vs. shared memory, in: *Hybrid Metaheuristics: Second International Workshop, HM 2005, Barcelona, Spain, August 29-30, 2005. Proceedings 2*, Springer, 2005, pp. 1–11.
- [32] B. Yu, Z. Yang, B. Yao, A hybrid algorithm for vehicle routing problem with time windows, *Expert Syst. Appl.* 38 (1) (2011) 435–441.
- [33] M. Alweshah, M. Almiani, N. Almansour, S. Al Khalailah, H. Aldabbas, W. Alomoush, A. Alshareef, Vehicle routing problems based on Harris Hawks optimization, *J. Big Data* 9 (1) (2022) 42.
- [34] S. Voigt, A review and ranking of operators in adaptive large neighborhood search for vehicle routing problems, *European J. Oper. Res.* 322 (2) (2025) 357–375.
- [35] A.P. Châu, L. Nguyen, B. Vo, Benchmark results for VRP with parcel locker, 2025, <http://dx.doi.org/10.17632/8xpyf25c7y.1>.
- [36] D.R. Jones, M. Schonlau, W.J. Welch, Efficient global optimization of expensive black-box functions, *J. Global Optim.* 13 (1998) 455–492.